GUIDE 64 · OPERATE

Try the Rust data plane in front of /q/

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=64-try-the-rust-data-plane. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** the data plane on your public port, serving your saved-query reads itself, Python behind it doing everything else, and a way to see which calls it served.

The data plane is a second process, `queryapigate-dataplane`, in the Docker image beside Python. For the calls it serves, it measured 3 to 9 times the calls a second per core of Python on its own, with p99 2 to 20 times lower (Performance, v5). It is **experimental** (ADR 0028, ADR 0030): what it serves, its settings and its headers may change in a minor release. It isn't in the PyPI package, and it isn't for a deployment that needs FIPS 140 - its cryptography isn't a validated module.

What it serves, and what it leaves to Python

On a server with OpenTelemetry tracing (`OTEL_EXPORTER_OTLP_ENDPOINT`) set, it serves nothing, and none with `QUERYAPIGATE_RATE_LIMIT` unless Redis is set too (below): it forwards every call. Otherwise it serves a call itself when all of these hold, and forwards it to Python, unchanged, when any doesn't:

- a `GET` or `POST` to `/q/<name>`, answered as JSON (no `?format=`, no streaming);
- with a **scoped API key** in the `default` workspace, with no quota or `max_concurrent` of its own, and a rate limit only with Redis (below). Its own `query_timeout` and `max_rows` are applied as Python applies them. Administrators, signed-in users, MCP and other workspaces always go to Python;
- on a **PostgreSQL or MySQL** connection, for a read-only saved query with bound parameters (no `{name}` text placeholders, no `embed` parameters, no response cache of its own);
- behind trusted proxies (`QUERYAPIGATE_TRUST_PROXY`), with `X-Forwarded-For` and `X-Forwarded-Proto` it reads exactly as Python does: one line each, no quotes or empty items, `http` or `https`. A proxy's ordinary headers are;
- and Python has answered the **same call with the same key once already**. That first call gives the data plane its plan: the SQL to send, the masking to apply, and digests of the key, query and connection rows it was made from. Every later call reads those rows again and is served only while they're unchanged.

Everything else - the Console, the Management API, writes, MCP, `/events`, `/metrics` - passes through to Python as if the data plane weren't there. Every run the data plane serves is still recorded in run history, the event log and `/metrics`.

Rate limits, with Redis

With `QUERYAPIGATE_REDIS_URL` set, the data plane serves keys with a `rate_limit` and servers with `QUERYAPIGATE_RATE_LIMIT` too (ADR 0032). It counts them in Python's own buckets in Redis, with Python's script, so a key has one budget whichever process takes its call, across every pod. Its 429s are Python's, header for header, and they count in `/metrics` and the rate-limit alerts as Python's do.

- **Without Redis, limited calls stay Python's.** Each process would otherwise count its own budget.
- **If Redis fails,** the data plane forwards limited calls to Python for 5 seconds at a time. Python counts them in Redis, or in its own memory with its *Rate limits are counted per instance* alert. A limit is never lifted.

- **It reads `redis://`, `rediss://` and `unix://` URLs without a query string or fragment.** For any other, limited calls stay Python's. It verifies a `rediss://` server's certificate and host name against the system's CAs.
- **Its Redis user can be one that only counts rate limits:** the script, and the commands the script runs, on `qag:ratelimit:*`. For example: `ACL SETUSER dataplane on >secret ~qag:ratelimit:* +evalsha +script|load +time +hmget +hset +expire`, and `+select` if the URL names a database other than 0. Python's user needs more: it also keeps the response cache, quotas and concurrency slots there.
- **Each limit counted is one Redis round trip a call:** about 50 µs on loopback, measured, plus the network's own round trip to Redis. A limited key still served 4.4 times Python's calls a second per core (measured).

1. With the Helm chart

One value: each API server pod gets a second container from the same image, which takes the pod's port, and Python moves to the pod's loopback. The chart makes the secret the two share, and keeps it across upgrades.

```yaml
dataplane:
  enabled: true
  resources:
    requests: {cpu: 500m, memory: 64Mi}
    limits: {cpu: "2", memory: 256Mi}   # about 2 CPUs a pod without a run store - see "What to watch for"
```

```shell
helm upgrade --install qag oci://ghcr.io/anantharajuc/charts/queryapigate -f values.yaml
kubectl logs deploy/qag-queryapigate -c dataplane   # queryapigate-dataplane 0.48.0: listening on 0.0.0.0:5000, ...
```

Behind the chart's ingress it serves as above: the chart sets `QUERYAPIGATE_TRUST_PROXY=1`, and the data plane reads the controller's `X-Forwarded-For` as Python does. With `helm template` or Argo CD, which can't see a Secret already in the cluster, give your own: `dataplane.existingSecret`, with the secret (at least 16 characters) under `dataplane.key`.

2. With Docker

Two containers from the same image, the second sharing the first one's network: Python on `127.0.0.1:5001`, the data plane on port 5000. Both need the same secret, settings and store.

```shell
export QUERYAPIGATE_DATAPLANE_SECRET="$(openssl rand -hex 32)"   # at least 16 characters; the same for both
IMAGE=ghcr.io/anantharajuc/queryapigate:0.48.0
docker run -d --name qag -p 5000:5000 -v qag-data:/data --env-file qag.env \
    -e QUERYAPIGATE_DATAPLANE_SECRET -e QUERYAPIGATE_BIND=127.0.0.1:5001 "$IMAGE"
docker run -d --name qag-dataplane --network container:qag -v qag-data:/data --env-file qag.env \
    -e QUERYAPIGATE_DATAPLANE_SECRET -e QUERYAPIGATE_DATAPLANE_LISTEN=0.0.0.0:5000 "$IMAGE" queryapigate-dataplane
```

`qag.env` holds your other settings, as before. The port is published on the first container, and the image's health check reaches both: in the shared network, port 5000 is the data plane, which passes `/health` to Python. In Compose, the second service takes `network_mode: service:<the first>`; for the scale-out example, add `deploy/scale-out/docker-compose.dataplane.yml`:

```shell
docker compose -f docker-compose.yml -f docker-compose.dataplane.yml up -d
```

3. See it serving

Call the same saved query a few times with a scoped key:

```shell
for id in 7 8 9; do curl -s -H "X-API-Key: $KEY" "http://localhost:5000/q/customer?id=$id"; sleep 1;
done
```

The answers are byte for byte what Python would send. Python answers the first, which gives the data plane its plan, and calls go on reaching Python until it has recorded the key's use, within one flush interval (`QUERYAPIGATE_HISTORY_FLUSH_INTERVAL`, 1 s). After that they show up in the **data plane's** log, not Python's: an `Executing on ...` line and the request line, in Python's format. Their runs appear in run history and the Console's live feed within a flush interval, and `/metrics` counts them as Python would.

If a call you expect to be served keeps going to Python, check it against the list above: an administrator's key, a key with a rate limit on a server without Redis, `?format=csv` or a text placeholder are the usual reasons. A data plane from another release than Python's serves nothing, and Python logs why once: in the image, both come from one build.

Its settings

SETTING	DEFAULT	WHAT IT DOES
`QUERYAPIGATE_DATAPLANE_SECRET`	none: required	Shared with Python. Without it, Python gives no plans and the data plane can't start
`QUERYAPIGATE_DATAPLANE_LISTEN`	`127.0.0.1:5000`	Where clients connect
`QUERYAPIGATE_DATAPLANE_UPSTREAM`	`http://127.0.0.1:5001`	The Python server
`QUERYAPIGATE_DATAPLANE_THREADS`	the CPUs it may use	Tokio worker threads; a container's CPU limit counts
`QUERYAPIGATE_DATAPLANE_POOL_SIZE`	16	Connections per database it serves from, on top of Python's pool
`QUERYAPIGATE_BIND`	`0.0.0.0:5000` in the image	Where the image's gunicorn listens: `127.0.0.1:5001` behind the data plane

It also reads Python's own settings that change an answer or a log line, and applies them as Python does: `QUERYAPIGATE_API_KEY`, `QUERYAPIGATE_SECRET_KEY`, `QUERYAPIGATE_MASK_KEY`, `QUERYAPIGATE_QUERY_TIMEOUT`, `QUERYAPIGATE_MAX_PAGE_SIZE`, `QUERYAPIGATE_METADATA_CACHE_TTL`, `QUERYAPIGATE_HISTORY_FLUSH_INTERVAL`, `QUERYAPIGATE_LOG_REQUESTS`, `QUERYAPIGATE_ACCESS_LOG`, `QUERYAPIGATE_JSON_LOGS`, `QUERYAPIGATE_SLOW_QUERY_THRESHOLD`, `QUERYAPIGATE_HISTORY_ADHOC_SQL`, `QUERYAPIGATE_TRUST_PROXY`, `QUERYAPIGATE_HSTS_MAX_AGE`, `QUERYAPIGATE_RATE_LIMIT` and `QUERYAPIGATE_REDIS_URL`. The metadata cache is where much of its speed comes from: with `QUERYAPIGATE_METADATA_CACHE_TTL=5`, a call reuses the key, query and connection rows it read in the last 5 seconds instead of reading the store, and a revoked key keeps working for up to 5 seconds, as it does on Python alone. With several pods, each caches on its own.

4. Built from the repository

Without the image: you need Rust (`dataplane/rust-toolchain.toml` names the version, and `rustup` installs it), and the same environment, user and store for both processes.

```shell
git clone https://github.com/AnanthaRajuC/QueryAPIGate && cd QueryAPIGate
cargo build --release --manifest-path dataplane/Cargo.toml
QUERYAPIGATE_BIND=127.0.0.1:5001 gunicorn --workers 1 --worker-class gthread \
    -c python:queryapigate.gunicorn_conf 'queryapigate.app:create_app()' &
QUERYAPIGATE_DATAPLANE_LISTEN=0.0.0.0:5000 dataplane/target/release/queryapigate-dataplane
```

Build it from the same commit as the Python you run: Python serves plans only to a data plane of its own release. Keep `127.0.0.1:5001` unreachable from outside the host: a request that reaches Python directly skips the data plane, which is harmless, but the data plane should be the only way in.

To stop using it

In the chart, `dataplane.enabled: false`. With Docker, stop the second container and start the first without `QUERYAPIGATE_BIND`. Nothing is stored on the data plane's side: plans live in its memory only.

What to watch for

- **Runs it can't record are dropped and counted** in `queryapigate_history_runs_total{outcome="dropped"}`. One Python worker records about 17,000 runs a second, and a data plane serves about 9,000 calls a second per CPU. Without a run store, keep a pod's data plane near 2 CPUs and add pods past that - held to 2, it recorded every one of 9.8 million runs at 16,400 calls a second (measured); with one (guide 65), the data plane writes its runs there itself and there's no such limit. More workers per pod would lift it too, but `/metrics` is kept per worker, so a scrape would see one of them.
- **The run history and event log grow with its rate.** The event log keeps every run for `QUERYAPIGATE_EVENTS_RETENTION_HOURS`; at tens of thousands of calls a second, that is more than a SQLite store should hold. Shorten it, or keep runs in ClickHouse.
- **More database connections:** each data plane opens up to `QUERYAPIGATE_DATAPLANE_POOL_SIZE` to each database it serves from, beside Python's pool. Count them per pod against the database's limit.
- **Writes and other forwarded calls pay one extra hop:** about 0.35 ms at p50 on the benchmark laptop.